

The Crowdsourced Festival Stage Optimizer

A 100% Autonomous, ML-Integrated Event Data Engine

Anushka Srivastava | Reg. No: 25BCE5388

Replacing the Human DJ

The Problem: Traditional Setup



Human Guesswork

DJs rely on observation to read a crowd's energy, which is subjective and prone to error.



Static Playlists

Pre-planned sets cannot pivot quickly if the crowd is unresponsive.

Our Solution: Autonomous Engine



Live Crowd Voting

Attendees use phones to upvote or downvote the current genre in real-time.



The Database

The web server has no brain. The PostgreSQL database makes all live decisions autonomously.

Data Flow



1. Offline ML (Python)

Pre-groups 1M songs into "Vibe Clusters" using K-Means clustering before the event.



2. The Messengers

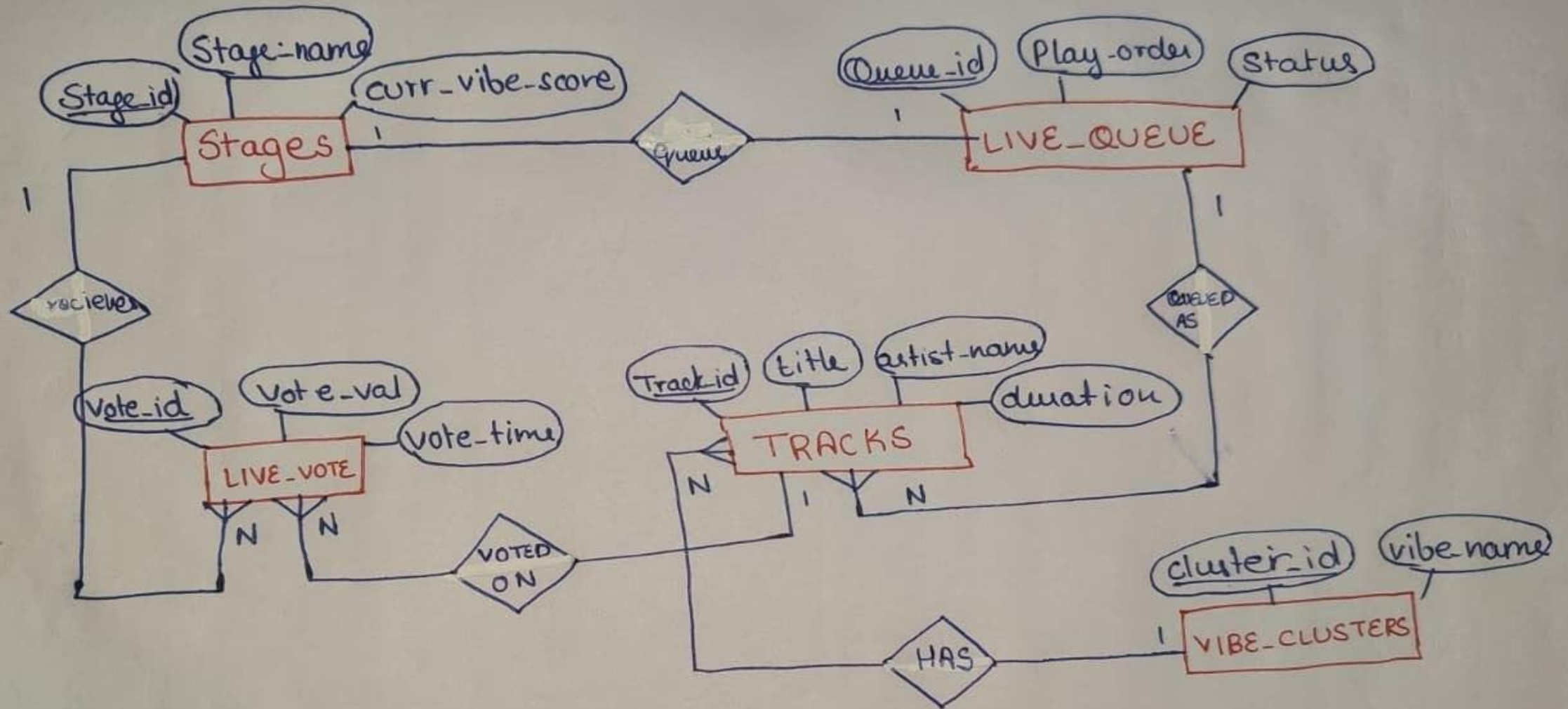
The frontend plays music; the backend blindly passes thousands of live votes to the DB.



3. The Brain (Postgres)

Catches votes, calculates scores, and rewrites the playlist completely autonomously.

Entity Relational Model



Enhanced Entity Relational Model

Anushka Srivastava → 25BCE5388

Stages

Attribute	Datatype	Constraint
→ Stage-id	NUMBER(5)	PRIMARY KEY
Stage-name	VARCHAR2(30)	NOT NULL
Curr-vibe-score	NUMBER(3)	NOT NULL

Live_Votes

Attribute	Datatype	Constraint
vote-id	NUMBER(10)	PRIMARY KEY
→ Stage-id	NUMBER(5)	FOREIGN KEY
→ track-id	NUMBER(10)	FOREIGN KEY
vote-val	NUMBER(3)	NOT NULL
vote-time	TIMESTAMP	NOT NULL

Live_Queue

Attribute	Datatype	Constraint
Queue-id	NUMBER(5)	PRIMARY KEY
→ Stage-id	NUMBER(5)	FOREIGN KEY
→ track-id	NUMBER(10)	FOREIGN KEY
play-order	NUMBER(3)	NOT NULL
Status	NUMBER(1)	NOT NULL

Tracks

Attribute	Datatype	Constraint
→ track-id	NUMBER(10)	PRIMARY KEY
→ cluster-id	NUMBER(5)	FOREIGN KEY
title	VARCHAR2(30)	NOT NULL
artist-name	VARCHAR2(30)	NOT NULL
duration	NUMBER(10)	NOT NULL

VIBE-CLUSTERS

Attribute	Datatype	Constraint
→ cluster-id	NUMBER(5)	PRIMARY KEY
vibe-name	VARCHAR2(30)	NOT NULL

Concurrency Control

The Threat: "Lost Updates"

During heavy vote spikes, thousands of users may submit votes in the exact same millisecond. Without strict concurrency control, simultaneous read/write operations will overwrite each other, causing the database to silently lose critical voting data.

The Fix: Two-Phase Locking

Implementation of strict transaction isolation levels and **Two-Phase Locking (2PL)**.

The DB acts as a traffic controller: Phase 1 acquires locks on the voting rows, executes the math, and Phase 2 releases them, guaranteeing compliance under stress.

Database Automation

How PostgreSQL replaces the human DJ using internal logic.



1. The Tripwire

PL/pgSQL Triggers actively monitor the 'Live_Votes' table. If a stage's aggregated vibe score drops below -50, the alarm is tripped.



2. The Recipe

The trigger automatically executes a **Stored Procedure**. It calculates the failing cluster and selects an adjacent, higher-energy vibe to pivot to.



3. The Execution

The procedure autonomously runs 'DELETE' and 'INSERT' commands to rewrite the 'Live_Queue'. The frontend simply plays the newly generated tracks.

The Crowdsourced Festival Stage Optimizer

A 100% Autonomous, ML-Integrated Event Data Engine

Anushka Srivastava | Reg. No: 25BCE5388
